

Continuous and Adaptive Learning over Big Streaming Data for Network Security

Pavol Mulinka[†], Pedro Casas*, Juan Vanerio[‡]

*AIT Austrian Institute of Technology, [†]CTU Czech Technical University in Prague, [‡]Universidad de la República

Abstract—Continuous and adaptive learning is an effective learning approach when dealing with highly dynamic and changing scenarios, where concept drift often happens. In a continuous, stream or adaptive learning setup, new measurements arrive continuously and there are no boundaries for learning, meaning that the learning model has to decide how and when to (re)learn from these new data constantly. We address the problem of adaptive and continual learning for network security, building dynamic models to detect network attacks in real network traffic. The combination of fast and big network measurements data with the re-training paradigm of adaptive learning imposes complex challenges in terms of data processing speed, which we tackle by relying on big data platforms for parallel stream processing. We build and benchmark different adaptive learning models on top of a novel big data analytics platform for network traffic monitoring and analysis tasks, and show that high speed-up computations (as high as $\times 6$) can be achieved by parallelizing off-the-shelf stream learning approaches.

Index Terms—Stream Machine Learning; Network Security; Big-Data.

I. INTRODUCTION

Machine learning for network security is a flourishing and active research domain, which has largely increased in the last decade; however, the general approach in the literature defines the problem as an off-line learning task, where machine learning models are trained once using labeled datasets, and are then applied as decision filters to the newly incoming measurements. This is a very restrictive approach when operating in real, highly dynamic environments, where concept drift - i.e., the changes in the underlying statistics of the learning/prediction target, occurs often and previous knowledge rapidly becomes obsolete.

We therefore investigate an adaptive, stream-based approach for machine-learning-based network security, training and benchmarking different machine learning algorithms for the analysis of continuously evolving network measurements. Stream machine learning analysis consists of processing a data instance at a time, keeping it in memory for a short period of time, and as such, using a limited amount of memory for learning. We implement different adaptive learning models, which take into account concept drifts to periodically re-train the underlying machine learning model. We evaluate their detection performance for multiple network attack types, using in-the-wild network measurements collected at the operational WIDE backbone network, and using the well-known MAWI-Lab labeled datasets [2]. We additionally describe the implementation and deployment of these stream machine learning approaches into Big-DAMA [3], a novel big data analytics

platform for large scale data analysis, benchmarking multiple models and different job parallelization and distribution strategies. Our results not only show that some stream-based machine learning models such as adaptive random forest are able to keep up with important concept drifts in the underlying network data streams, but also that high speed up computations can be achieved by parallelizing stream machine learning approaches on distributed monitoring platforms.

II. RELATED WORK & STREAM LEARNING BASICS

Machine learning has been largely applied into network security and anomaly detection problems in the literature [1], mostly relying on off-line based learning. However, the application of stream machine learning is significantly less popular; close to this paper, authors evaluate stream-based traffic classification approaches based on Hoeffding Adaptive Trees in [4], relying on MAWILab data and using MOA (Massive Online Analysis) machine learning libraries - as we do here.

When it comes to the data stream learning domain, there are multiple interesting papers to consider, covering general issues related to the learning properties of stream-based algorithms [6], [7], the analysis of large-scale datasets [8], the analysis of model evaluation issues [9] as well as proposals of general stream-learning frameworks for the analysis of continuous data [10]. Learning with *imbalanced classes* and under the presence of *concept drifts* are two complex problems which normally affect stream learning, and both are deeply covered in [11].

In this paper we consider a continually adaptive memory length, window-based approach known as ADWIN (ADaptive WINdowing) [12]. In a nutshell, ADWIN maintains a memory window of variable size containing past samples which are used to re-train the underlying model, based on the detection of concept drifts. Using ADWIN, we conduct prequential evaluation of k -fold cross-validated (CV) stream models [13]. Prequential evaluation uses each instance first to test an initial or bootstrapped model, and then to train or update the corresponding model. We implement and benchmark four stream-based machine learning algorithms through ADWIN prequential k -fold CV, comparing their performance to the corresponding off-line versions of the models. This paper extends our recent work on stream-based traffic analysis [5], benchmarking the proposed algorithms in a distributed computing platform, and further analyzing the underlying concept-drift events conditioning the learning performance.

III. STREAM-BASED MACHINE LEARNING

In most domains, input data evolve over time. This could result in poor detection performance in learning models which assume a static relationship between input and output variables. Concept drift refers to this phenomenon where the underlying relationships in the data change over time [14]. There are multiple strategies to deal with concept drift. A relevant one when dealing with highly dynamic environments is to periodically re-train the learning model with recent historical data kept in memory, as soon as a major change in the data distribution is detected. Abrupt changes can be handled by reactive strategies. A reactive learning strategy consists of a sliding window keeping the most recent historical data for re-training, but additionally adding a change-detection algorithm to rapidly and automatically identify concept drifts, discarding “irrelevant” past data. We implement a reactive learning strategy following these principles through ADWIN, a dynamically adjusting window-based approach introduced in [12], which maintains a window of variable size containing training samples. The algorithm automatically grows the window when no change is apparent, and shrinks it when the statistical properties of the stream change, using a simple mean-based change detector. ADWIN automatically adjusts its window size to the optimum balance point between reaction time and small variance; using ADWIN, we implement four stream-based machine learning algorithms popular in the literature, including incremental K -NN, Hoeffding Adaptive Trees (HAT), Adaptive Random Forests (ARF) and Stochastic Gradient Descent for SVM (SGD).

A. ADWIN Adaptation Strategy

The ADWIN algorithm keeps a sliding window W with the most recently observed measurements x_i . At each time t , instance x_t is generated according to an unknown probability distribution D_t . Let μ_t be the expected value of x_t when it is generated according to D_t . Naturally, μ_t is unknown for all t . Let n be the length of W , $\hat{\mu}_W$ the (computed) average of the measurements in W , and μ_W the (unknown) average of μ_t for $t \in W$. The idea of ADWIN is straightforward: whenever two *large enough* sub-windows of W exhibit *distinct enough* averages, we conclude that the corresponding expected values are different, and the older portion of the window is dropped. More specifically:

- 1: initialize window W
- 2: **for** each $t > 0$ **do**
- 3: $W \leftarrow W \cup \{x_t\}$ (add x_t to the head of W)
- 4: **repeat** drop instances from the tail of W
- 5: **until** $\|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}\| \geq \epsilon$ for every split of $W = W_0 \cdot W_1$
- 6: return $\hat{\mu}_W$
- 7: **end for**

where $\hat{\mu}_{W_0}$ and $\hat{\mu}_{W_1}$ are the averages of the instances in W_0 and W_1 respectively. ADWIN is therefore a simple statistical test for different distributions in W_0 and W_1 , which checks whether the observed average in both sub-windows differs by more than the threshold ϵ .

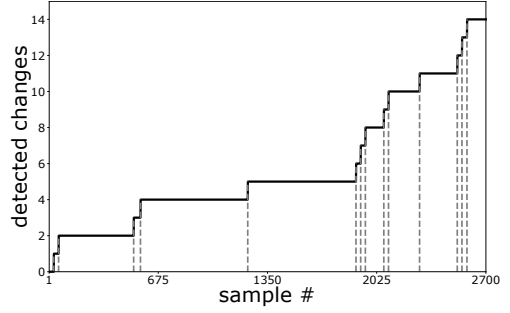


Figure 1: *Page-Hinkley Concept Drift Detection*. Changes in the dataset distribution detected by the Page-Hinkley test. Detected changes are marked with dashed lines.

B. Concept Drift Detection

Different change detection algorithms can be applied to identify the times when the probability distribution of a stochastic process or time series changes. In our problem, such detection has to be performed in an on-line manner, i.e., without assuming that the statistics of the complete time series are known in advance. The ADWIN algorithm is by conception an on-line change-detection algorithm. In this paper we consider an additional change detection algorithm to analyze the considered dataset: the Page-Hinkley Test (PHT). PHT is a standard statistical test for change detection, commonly used in time series analysis.

IV. EVALUATION RESULTS

To evaluate the performance of the proposed algorithms and adaptation strategy, we consider the detection of diverse types of network attacks in real network traffic measurements collected at the WIDE backbone network, using MAWILab labeled datasets. MAWILab is a public collection of 15-minute network traffic traces captured every day on a backbone link between Japan and the US since 2001 [2].

We evaluate the proposed adaptive algorithms following a new strategy to evaluate stream-based algorithms [13], using prequential k -fold cross validation; the strategy is basically an adaptation of cross-validation to the distributed streaming setting, and assumes we have k different instances of the corresponding model we want to evaluate, running in parallel. Each time a new sample arrives, it is used for testing in one of the k models selected randomly, and then used for training by all the other models. As evaluation metric, we take the model detection accuracy (ACC).

A commonly applied approach to evaluate the performance of stream-based algorithms is to benchmark them against their corresponding off-line, batch implementations. Therefore, we use as baseline the results obtained for the corresponding batch-based algorithms, including K -NN, Hoeffding Tree (HT-batch), Random Forest (RF-batch) and SVM/SGD (SGD-batch). We subtract the batch metric results from prequential results to show the performance of stream algorithms in comparison to their batch versions. We use MOA machine learning libraries to perform the analysis, including both the parameters calibration (grid-based search) and the models

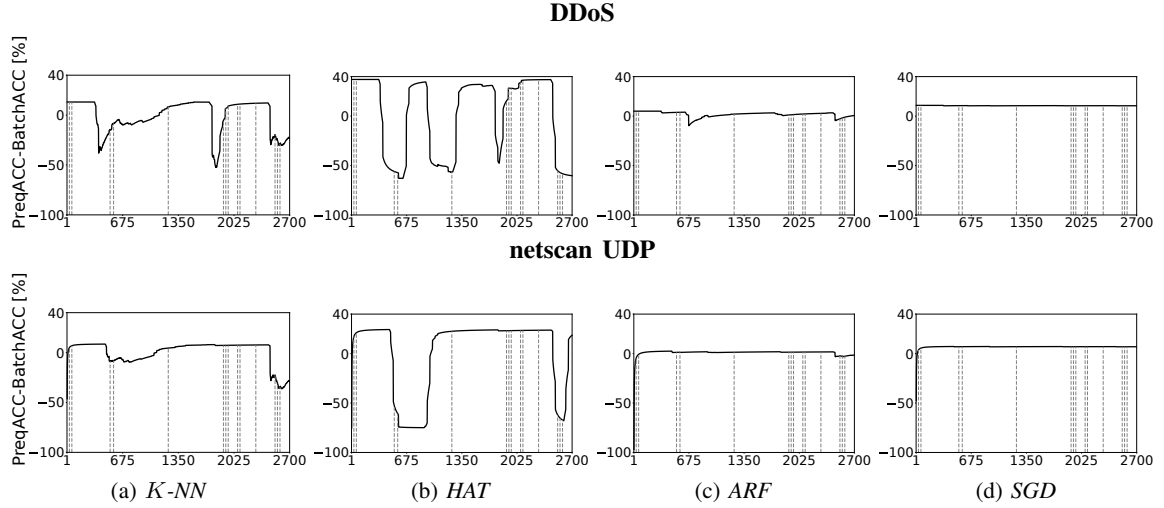


Figure 2: *Prequential 10-fold Cross Validation Accuracy Evaluation*. Diagrams show Prequential 10-fold CV results for each algorithm for each attack type. Concept drifts detected by the Page-Hinkley test are marked with dashed lines.

training and evaluation. MOA is specifically designed for stream-based machine learning approaches.

A. Data Description

The traffic studied in this paper spans 15 days of 15'-packet traces collected in late 2015. From the labeled anomalies and attacks, we focus on those which are detected simultaneously by the four MAWILab detectors, using in particular the ones labeled as “anomalous” by MAWILab. We consider 5 types of attacks/anomalies: (i) DDoS attacks, (2) HTTP flashcrowds, (3) Flooding attacks, and two different flavors of distributed network scans (netscan) using (4) UDP and (5) TCP-ACK probing traffic. We train different models to detect each of these attack types separately, thus each detection approach consists of five different binary detectors.

To detect these attacks, we consider a slotted, time-based evaluation. For doing so, we split the traffic traces in consecutive time slots of 5 seconds each, and compute a set of features describing the traffic in each of these slots. The total number of samples is therefore $900/5$ (15' per day) $\times 15$ (days) = 2700. In addition, each slot i is assigned a label l_i , consisting of a binary vector $l_i \in \mathbb{R}^{5 \times 1}$ which indicates at each position if anomaly of type $j = 1..5$ is present or not in current time slot. We compute a large number n of features describing a time slot, derived from the empirical distributions of traditional packet measurements including traffic throughput, packet sizes, IP addresses and ports, transport protocols, flags, etc. The total set accounts for $n=245$ features, which are computed for every time slot $i = 1..m$.

B. Performance Evaluation

We start by analyzing the variation of the statistical properties of the considered dataset, detecting concept drifts with the PHT test. Fig. 1 depicts the cumulative number of changes observed in the dataset, as well as the times when those changes are detected. The test detects 14 abrupt changes during the

total measurements time span. Note that the frequency of changes significantly increases in the last third of the dataset, with more than 10 changes detected in the last 4 days.

We then evaluate the performance of the algorithms in five binary classification variants - one for each attack type, resulting in five sets of results for each tested algorithm. For the sake of space limitations, Fig. 2 reports the performance results for only two attack types - DDoS and netscan using UDP traffic, considering detection accuracy as performance metric, and using the batch-algorithms accuracy as baseline. Similar results are observed for the other attack types.

The prequential 10-fold CV performance evaluation shows that both the ARF and the SGD models rapidly converge to the batch-based accuracy results, with minimum performance variations under concept drifts, and with a slightly better performance for the SGD model, this one even outperforming the batch-based performance. On the other hand, both the K -NN and HAT models provide results which tend to oscillate around the batch algorithm baseline. In the case of HAT we can appreciate the correlation between the detected concept drifts and the performance variations of the model. Interestingly, the HAT model is the one achieving the highest accuracy of the four models – up to 40% improvement w.r.t. the baseline, but also poor performance under concept drifts.

V. DISTRIBUTED STREAM MACHINE LEARNING

The final question we address is scalability of the analysis. We deploy the used MOA libraries on top of Big-DAMA [3] – <https://bigdama.ait.ac.at/>, a big data platform we have conceived for the specific task of network monitoring and analysis. We take a step towards the unification of distributed machine learning frameworks which are unfortunately being abandoned such as SAMOA or distributed Weka over Spark. The main advantage of integrating machine learning libraries such as Weka and MOA rely on the great diversity of algo-

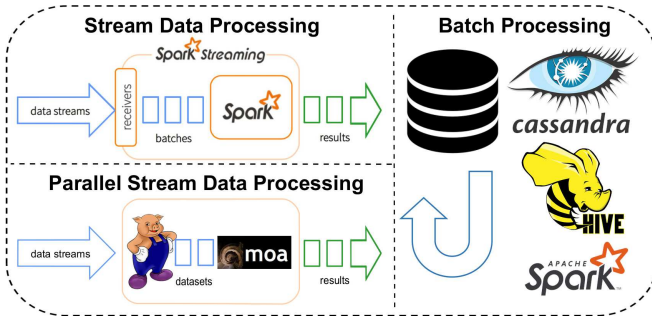


Figure 3: Combined stream and batch-based architecture for big data analytics, using a Hadoop ecosystem.

rithms and models available in such libraries, seldom available in distributed-nature libraries such as spark ML or MLlib.

Big-DAMA uses off-the-shelf big data storage and processing engines to offer both stream data processing and batch processing capabilities, following a standard lambda architecture, decomposing separate frameworks for stream, batch and query. This approach to architecture attempts to balance latency, throughput, and fault tolerance by using batch processing to provide comprehensive views of batch data, while simultaneously using real-time stream processing to provide on-line data analysis capabilities. Fig. 3 shows a high level architectural design of the platform. It uses Apache Spark streaming for stream-based analysis, Apache Pig for parallel distributed computation, Spark for batch analysis, and Apache Cassandra and Hive for query and storage. Big-DAMA is currently deployed on top of a virtualized data cluster, consisting of 12 virtual nodes with a total capacity of 650 GB of memory and 40 TB of data storage.

We integrate the presented approaches on top of this platform, and compare the execution times involved in the full analysis of the studied dataset over two different scenarios: a local, one single node scenario, where all MOA jobs are executed on a single node of the cluster, and a fully distributed scenario, where all MOA jobs are assigned to the 12-node distributed cluster. The integration of MOA within the distributed cluster is done through Apache Pig, which allows to define standard Python-based User Defined Functions (UDFs) to run external code on top of Map-Reduce generated programs.

The total number of MOA jobs covering the study is 20: there are 5 attack-types and 4 stream-based models – the 10-fold cross validation for ADWIN is done in parallel, so we count it as a single job. Fig. 4 reports the comparative execution times for five different variations of the analysis: variations 1, 2 and 3 referred to as LS (Local-Sequential) correspond to the local, one node execution of the 20 analysis jobs in a sequential manner, running one job after the other. v1-LS corresponds to the standard MOA execution, v2-LS considers the porting of MOA Java libraries into Jython (Python on Java), and v3-LS includes the integration of MOA into Apache Pig, through Python UDFs. Variations 4 and 5 correspond to the distributed execution of the 20 MOA jobs on top of the 12 nodes - using Pig, with v4-DS being the

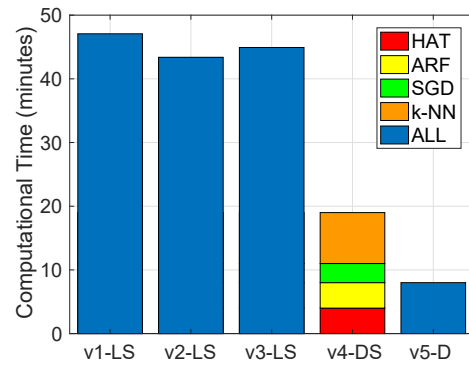


Figure 4: Computational time in processing MAWILab data.

fully distributed yet sequential execution, and v5-D a fully distributed and parallelized execution.

The benchmark shows two interesting facts: firstly, that there is no apparent overhead introduced by running MOA Java libraries on top of Apache Pig - v1/2/3 have the same execution times; secondly, a fully distributed and parallelized execution of the stream-based machine learning models can reduce execution times by a factor of up to 6. Let us consider this performance increase from an overall perspective. Stream-based algorithms are already, by conception, way faster than their corresponding off-line, batch-based versions. So besides the performance increase introduced by the stream-based paradigm, we can further reduce computational times, processing 2 weeks of measurements with 20 jobs in about 7 minutes. To conclude, note that the results obtained through v4-DS reveal that the bottleneck task is the k -NN evaluation, which basically bounds the execution time of the fastest v5-D variation to the the slowest evaluation task.

We are now working on the comparison of Big-DAMA against other streaming frameworks such as Apache Flink distributed stream-processing, as well as the SAMOA framework for stream-based massive on-line data analytics.

REFERENCES

- [1] R. Boutaba et al., "A Comprehensive Survey on Machine Learning for Networking: Evolution, Applications and Research Opportunities," *Jour. of Internet Services and Applications*, vol. 9, no. 1, 2018.
- [2] R. Fontugne et al., "Mawilab: combining diverse anomaly detectors for automated anomaly labeling and performance benchmarking," in *ACM CoNEXT*, 2010.
- [3] P. Casas et al., "Network security and anomaly detection with big-dama, a big data analytics framework," in *IEEE CloudNet*, 2017.
- [4] V. Carrel-Español et al., "A streaming flow-based technique for traffic classification applied to 12+ 1 years of internet traffic," *Telecom. Sys.*, vol. 63, no. 2, 2016.
- [5] P. Mulinka et al., "Stream-based Machine Learning for Network Security and Anomaly Detection," in *Big-DAMA Workshop*, 2018.
- [6] P. M. Domingos and G. Hulten, "Catching up with the data: Research issues in mining data streams," in *DMKD*, 2001.
- [7] M. Stonebraker et al., "The 8 requirements of real-time stream processing," *ACM Sigmod Record*, vol. 34, no. 4, pp. 42–47, 2005.
- [8] G. Hulten et al., *Mining massive data streams*. University of Washington, 2005.
- [9] J. Gama et al., "Issues in evaluation of stream learning algorithms," in *ACM SIGKDD*, 2009.
- [10] —, "On evaluating stream learning algorithms," *Machine learning*, 2013.
- [11] T. R. Hoens et al., "Learning from streaming data with concept drift and imbalance: an overview," *Progress in Artificial Intelligence*, vol. 1, no. 1, pp. 89–101, 2012.
- [12] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proceedings of the 2007 SIAM Conference*, 2007, pp. 443–448.
- [13] A. Bifet et al., "Efficient online evaluation of big data stream classifiers," in *ACM SIGKDD*, 2015.
- [14] J. Gama et al., "A survey on concept drift adaptation," *ACM CSUR*, vol. 46, 2014.